The Journal of Open Source Software

ncpi: An integrated forward and inverse modeling toolkit for inferring neural circuit parameters from electrophysiological data

Laura Torres Soria [1*], **Alejandro Orozco Valero** [1*], **Marta Cárdenas Sánchez** [1], **Francisco Pelayo** [1,2], **Christian Morillas** [1,2], **and Pablo Martínez-Cañada** [1,2*]

1 Research Center for Information and Communication Technologies (CITIC), University of Granada, Granada, Spain **2** Department of Computer Engineering, Automation and Robotics, University of Granada, Granada, Spain * These authors contributed equally.

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Kevin M. Moerman

Submitted: 03 July 2026
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

Modern computational neuroscience now provides efficient tools for multiscale simulation of neural activity and increasingly powerful inference approaches for connecting these models to empirical data. Despite these advances, the mechanistic relationship between recorded population-level brain signals and the neural processes that generate them remains incompletely understood. Addressing this gap is a central challenge in both basic neuroscience and clinical neurophysiology, where a more mechanistic interpretation of neural signals could improve the identification, stratification, and monitoring of neurological and psychiatric conditions.

Simulators and inverse-modeling libraries provide many of the individual components required to interpret brain signals in terms of their underlying neural mechanisms. However, combining these components into a complete workflow typically requires substantial expertise in neural simulation, forward modeling, biomarker discovery, machine learning, data-format conversion, and statistical analysis. `ncpi` is an open-source Python toolbox designed to make this process more accessible to both computational specialists and clinical or neuroscience researchers by bringing these steps together within a unified and reproducible workflow.

The initial presentation of `ncpi` introduced the core software platform and demonstrated its use for model-driven interpretation of mouse local field potentials (LFP) data and human electroencephalography (EEG) recordings (Orozco Valero et al., 2025). Since then, the package has evolved substantially through improvements to the core API and the addition of two major capabilities: heterogeneous empirical dataset handling and a graphical web interface for configuring and running complete workflows. The dataset-handling functionality allows `ncpi` to load and parse electrophysiological data from different recording modalities, file formats, and experimental structures into a common tabular representation, enabling downstream analyses to be reused without rewriting modality- or dataset-specific code. In this way, `ncpi` supports both synthetic data generation and the analysis of user-provided electrophysiology datasets, including recordings at different spatial scales such as LFP, electrocorticography (ECoG), magnetoencephalography (MEG), and EEG. The graphical interface provides a user-friendly entry point for configuring and running simulation and empirical-data workflows, while the underlying Python API remains available for expert users who require full scriptable control. The Python components of `ncpi` are operating-system-independent (although some dependencies, such as `NEST`, require Windows users to install WSL), can run locally or on a server, and are supported by documentation including video-based tutorials and interactive Jupyter Notebooks.

DRAFT

The Journal of Open Source Software

Statement of need

A full mechanistic interpretation of electrophysiological recordings requires modeling both the forward and inverse pathways: how activity generated by neural microcircuits gives rise to measurable brain signals, and how those signals can be used to infer properties of the underlying neural processes. In practice, this requires integrating multiple steps into a complex workflow spanning neural simulation, time-series analysis, machine learning, and statistical analysis. A typical workflow may involve simulating spiking or multicompartmental neural networks with simulators such as `NEST` (Eppler et al., 2009) or `NEURON` (Hines & Carnevale, 1997), estimating extracellular potentials with tools such as `LFPy` and `LFPykernels` (Hagen et al., 2018, 2022), computing macroscale signals such as EEG using forward head models (Huang et al., 2016), extracting neurophysiologically relevant time-series features with packages such as `catch22`, `hctsa`, or `specparam` (Donoghue et al., 2020; Fulcher & Jones, 2017; Lubba et al., 2019), and fitting inverse models using tools such as `scikit-learn` or the simulation-based inference (`sbi`) toolbox (Tejero-Cantero et al., 2020). These tools are individually valuable, but they are not designed as a single interoperable workflow. As a result, researchers often face a steep learning curve, fragmented installation procedures, custom data-conversion requirements, and the need for substantial domain knowledge before they can run an end-to-end analysis.

`ncpi` addresses this gap by providing a unified software layer for forward simulation and inverse parameter inference from electrophysiological signals. It connects single-cell neural network simulation, biophysical field-potential computation, extraction of neurophysiologically relevant features, inverse modeling, empirical-data parsing, and statistical analysis within a consistent Python API and an accessible graphical interface Figure 1. By integrating these components into a single workflow, `ncpi` lowers the technical barrier for researchers interested in testing whether candidate electrophysiological biomarkers reflect specific neural circuit parameters, while preserving access to established backend libraries for users who require methodological control.

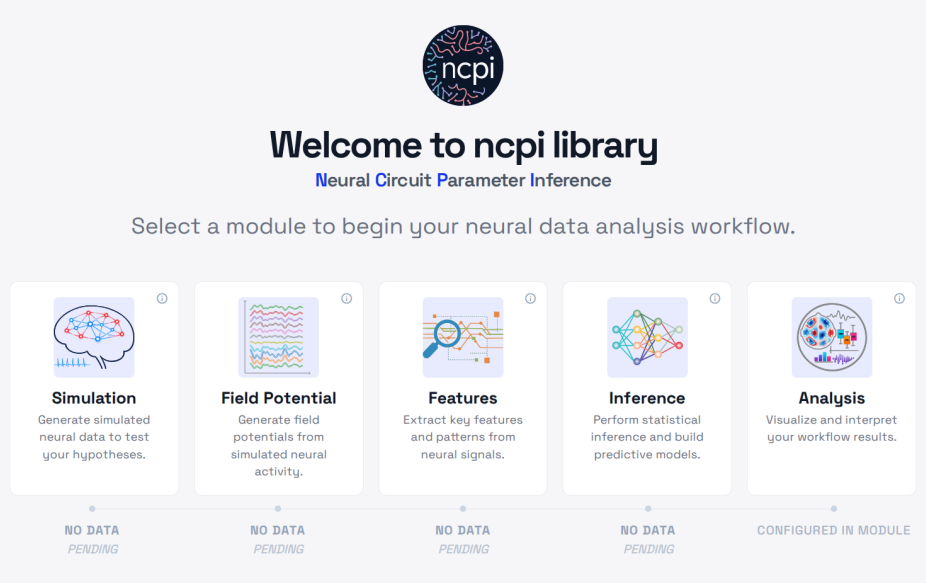


Figure 1: Overview of the graphical interface dashboard, showing the different modules that can be configured.

The toolbox is intended for computational neuroscientists, experimental neuroscientists, clinical neurophysiology researchers, and students seeking to link electrophysiological measurements to mechanistic models. It is particularly useful for comparing individual biomarkers and biomarker combinations, benchmarking inverse models on simulated data, applying trained models to

empirical LFP, ECoG, MEG, or EEG recordings, and rapidly inspecting how inferred circuit parameters vary across experimental groups, recording sites, brain states, or disease stages.

State of the field

A mature ecosystem of computational tools already supports many of the individual steps required for this type of mechanistic modeling workflow. `NEST` provides efficient simulation of large spiking networks (Eppler et al., 2009), while `NEURON` supports detailed multicompartment neuron modeling (Hines & Carnevale, 1997). `LFPy` and `LFPykernels` enable biophysical forward modeling of extracellular signals (Hagen et al., 2018, 2022). Feature libraries such as `hctsa` and `catch22` provide broad time-series phenotyping (Fulcher & Jones, 2017; Lubba et al., 2019), while `specparam` supports parametrization of periodic and aperiodic components in neural power spectra (Donoghue et al., 2020). `scikit-learn` and `sbi` (Tejero-Cantero et al., 2020) provide mature machine-learning and probabilistic inference tools.

The main limitation is not the absence of high-quality software but the lack of a connected, user-friendly workflow that spans all stages of the analysis. Users commonly have to install several packages with different assumptions, write custom scripts to move data between them, and understand modeling, signal-processing, and inference details before they can answer a biological question. `ncpi` was developed to fill this integration gap. It does not replace the underlying simulators, forward models, feature extractors, or inference engines; instead, it coordinates them through a coherent API, standardized data objects, parser utilities, example workflows, and a graphical interface.

Compared with single-purpose simulation or signal-analysis packages, `ncpi` encompasses all steps of the forward and inverse modeling pipeline for electrophysiological population signals. It is also distinct from custom analysis scripts accompanying individual studies because it exposes reusable classes, parser configuration, cross-platform installation options, and a `WebUI` for reproducible workflows across datasets.

Software design

`ncpi` follows three design principles: a user-friendly and modular object-oriented API, interoperability with established scientific software, and support for both code-based and graphical workflows. The core package is organized around classes that correspond to the main stages of the analysis: `Simulation` for executing neural circuit model scripts, `FieldPotential` for forward modeling of field potentials, `Features` for extracting neurophysiologically relevant time-series features, `Inference` for training and applying inverse models, `Analysis` for statistical analysis and visualization, and `EphysDatasetParser` for converting heterogeneous empirical recordings into a common schema.

The main design trade-off was to connect the full workflow while preserving visibility and control over the specialist tools used at each stage. `ncpi` therefore gives users an integrated workflow, so they do not need to manually glue together simulation engines, field-potential solvers, feature libraries, and inference methods. At the same time, it still exposes important settings and assumptions of each backend tool, allowing expert users to understand and control what is happening. This design reduces duplicated code and preserves methodological transparency, but it requires the package to manage differences in data formats, execution models, and dependency availability across tools. Optional dependencies are therefore loaded only by the components that need them, for example `NEST`-based simulations, `LFPy`/`LFPykernels` field-potential kernels, `lfpykit` M/EEG forward models, feature libraries such as `catch22`, `hctsa` and `specparam`, `scikit-learn` regressors, `sbi` density estimators, and R-backed statistical models. The class-based API was chosen to make these boundaries explicit: each class owns one stage of the workflow, while shared data conventions allow results to move between stages without ad hoc conversion scripts.

DRAFT

This modular structure allows users to run complete pipelines or use individual components independently. For example, a user can simulate a spiking network, compute proxy signals, current dipole moments, LFPs, or M/EEG signals, extract `catch22`, `hctsa` or spectral-parameterization features, train a `scikit-learn` regressor or an `sbi` model, and then apply the trained inverse model to empirical data. Alternatively, a user can bypass simulation and use only the parser, feature extraction, inference, or analysis modules on pre-existing data. Parallel execution is supported across computationally intensive stages, either through backend tools with native parallelism, such as `NEST`, or through multiprocessing-based parallelism implemented in `ncpi`.

The `EphysDatasetParser` is a major addition in the current release. It accepts arrays, pandas data frames, dictionaries, `MATLAB` files, `NumPy` files, `JSON`, `NWB`, `EDF`, `MNE`-compatible objects and files, `BrainVision`-style data, and tabular inputs, then maps them to canonical fields such as subject, group, condition, epoch, sensor, recording type, sampling frequency, signal data, time bounds, frequency-domain metadata, and source file. The trade-off in this parser is between accepting heterogeneous user data and requiring a stable internal representation for reproducible downstream analysis. Optional preprocessing such as temporal segmentation, z-scoring, epoching, and aggregation is applied before feature extraction, inference, or statistical analysis. This makes the package usable with arbitrary empirical datasets rather than only with example data distributed by the project.

The `WebUI` exposes the same conceptual workflow through a browser-based interface. It is organized into simulation, field-potential, feature, inference, and analysis modules. These modules support loading precomputed outputs or empirical datasets, running predefined and custom simulation workflows, computing proxy and kernel-based field-potential outputs, projecting current dipole moments to EEG/MEG sensors, extracting features, training inverse models, computing predictions, and plotting results. The interface is intentionally simple, with guided pages for each stage of the pipeline, and can be run locally or on a remote server through SSH tunneling. This supports its use on workstations and high-performance computing infrastructures. At the same time, the `WebUI` is built around the same conceptual stages as the Python API, so graphical workflows remain close to the scriptable implementation rather than forming a separate analysis path.

Research impact statement

`ncpi` has already been used to link empirical electrophysiological data to interpretable neural circuit parameters. Its first full software presentation, published in 2025 (Orozco Valero et al., 2025), introduced a two-million-sample simulation resource based on recurrent leaky integrate-and-fire networks, compared single- and multi-feature inverse models, and demonstrated applications to mouse developmental LFP recordings and human EEG data from Alzheimer's disease cohorts. This established `ncpi` as both a benchmarking platform for candidate electrophysiological biomarkers and a practical framework for model-based interpretation of population-level brain signals at different scales.

Since then, `ncpi` has supported studies of cortical dysfunction in Alzheimer's disease (Cárdenas Sánchez et al., 2026) and excitation/inhibition imbalance in young adults exposed to childhood interpersonal trauma (Orozco Valero et al., 2026).

Together, these applications illustrate the central purpose of the toolbox: to test whether measurable electrophysiological features can reveal circuit-level alterations across neurodegenerative, neurodevelopmental, and other clinical contexts.

DRAFT

AI usage disclosure

Generative AI tools assisted with selected aspects of software development, particularly the graphical interface (`WebUI`) components. All generated or AI-assisted software was subsequently reviewed by the authors. Generative AI tools were also used to support the writing of this manuscript, although the original content was provided by the authors. The manuscript content was fully reviewed and revised by the authors.

Acknowledgements

This study was supported by grants PID2022-139055OA-I00 and PID2022-137461NB-C31, funded by MCIN/AEI/10.13039/501100011033 and by "ERDF A way of making Europe"; and by grant RYC2024-049595-I funded by MCIN/AEI/10.13039/501100011033 and FSE+.

References

Cárdenas Sánchez, M., Orozco Valero, A., García, J. M., Rodríguez-González, V., Montobbio, N., Pelayo, F., Morillas, C., Poza, J., Gómez, C., & Martínez-Cañada, P. (2026). A Framework Integrating Spiking Cortical Circuit Modeling and Simulation-Based Inference to Probe Biomarkers of Cortical Dysfunction in Alzheimer's Disease. *Interdisciplinary Sciences: Computational Life Sciences*. https://doi.org/10.1007/s12539-026-00817-8

Donoghue, T., Haller, M., Peterson, E. J., Varma, P., Sebastian, P., Gao, R., Noto, T., Lara, A. H., Wallis, J. D., Knight, R. T., Shestyuk, A., & Voytek, B. (2020). Parameterizing neural power spectra into periodic and aperiodic components. *Nature Neuroscience*, *23*(12), 1655–1665. https://doi.org/10.1038/s41593-020-00744-x

Eppler, J. M., Helias, M., Muller, E., Diesmann, M., & Gewaltig, M.-O. (2009). PyNEST: a convenient interface to the NEST simulator. *Frontiers in Neuroinformatics*, *2*, 12. https://doi.org/10.3389/neuro.11.012.2008

Fulcher, B. D., & Jones, N. S. (2017). hctsa: A Computational Framework for Automated Time-Series Phenotyping Using Massive Feature Extraction. *Cell Systems*, *5*(5), 527–531.e3. https://doi.org/10.1016/j.cels.2017.10.001

Hagen, E., Magnusson, S. H., Ness, T. V., Halnes, G., Babu, P. N., Linssen, C., Morrison, A., & Einevoll, G. T. (2022). Brain signal predictions from multi-scale networks using a linearized framework. *PLOS Computational Biology*, *18*(8), 1–51. https://doi.org/10.1371/journal.pcbi.1010353

Hagen, E., Næss, S., Ness, T. V., & Einevoll, G. T. (2018). Multimodal Modeling of Neural Network Activity: Computing LFP, ECoG, EEG, and MEG Signals With LFPy 2.0. *Frontiers in Neuroinformatics*, *12*, 92. https://doi.org/10.3389/fninf.2018.00092

Hines, M. L., & Carnevale, N. T. (1997). The NEURON Simulation Environment. *Neural Computation*, *9*(6), 1179–1209. https://doi.org/10.1162/neco.1997.9.6.1179

Huang, Y., Parra, L. C., & Haufe, S. (2016). The New York Head–A precise standardized volume conductor model for EEG source localization and tES targeting. *NeuroImage*, *140*, 150–162. https://doi.org/10.1016/j.neuroimage.2015.12.019

Lubba, C. H., Sethi, S. S., Knaute, P., Schultz, S. R., Fulcher, B. D., & Jones, N. S. (2019). catch22: CAnonical Time-series CHaracteristics. *Data Mining and Knowledge Discovery*, *33*(6), 1821–1852. https://doi.org/10.1007/s10618-019-00647-x

Orozco Valero, A., Kopiś-Posiej, N., Rodríguez-González, V., Gutiérrez-de Pablo, V., Morillas, C., Poza, J., Gómez, C., Martínez-Cañada, P., & Krukow, P. (2026). Neurophysiological excitation/inhibition imbalance in young adults burdened with childhood interpersonal

DRAFT

trauma. *bioRxiv*. https://doi.org/10.64898/2026.01.14.699432

Orozco Valero, A., Rodríguez-González, V., Montobbio, N., Casal, M. A., Tlaie, A., Pelayo, F., Morillas, C., Poza, J., Gómez, C., & Martínez-Cañada, P. (2025). A Python toolbox for neural circuit parameter inference. *Npj Systems Biology and Applications*, *11*(1), 45. https://doi.org/10.1038/s41540-025-00527-9

Tejero-Cantero, A., Boelts, J., Deistler, M., Lueckmann, J.-M., Durkan, C., Gonçalves, P. J., Greenberg, D. S., & Macke, J. H. (2020). sbi: A toolkit for simulation-based inference. *Journal of Open Source Software*, *5*(52), 2505. https://doi.org/10.21105/joss.02505

DRAFT